Lab 2

1 Introduction

Welcome to the second programming lab! In this lab, we will explore 2D arrays (matrices) and functions in Python. We will go through examples and dry runs to solidify your understanding.

2 2D Arrays

A 2D array, also known as a matrix, is an array of arrays where each element is identified by two indices. It can be visualized as a table of rows and columns.

2.1 Creating a 2D Array

You can create a 2D array in Python using nested lists. Here's an example:

```
# Creating a 2D array
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Printing the 2D array
for row in matrix:
    print(row)
```

Listing 1: 2D Array Example

Dry Run:

- **Initialization:** `matrix` is a 2D array with 3 rows and 3 columns.
- **Iteration:** Loop through each row in `matrix`.
- **Output:** Print each row.

2.2 Accessing Elements

Elements in a 2D array can be accessed using row and column indices:

```
# Accessing elements
element = matrix[1][2]
print("Element at row 2, column 3:", element)
```

Listing 2: Accessing Elements in a 2D Array

Dry Run:

- **Access:** Access the element at row 2, column 3 (indexing starts from 0).
- **Output:** Print the accessed element.

PREPARED BY: Sahil Wagh(M.Tech),Nitish Dumoliya (Phd)

2.3 Matrix Initialization with User Input

In this subsection, we will learn how to initialize a matrix of zeros with dimensions specified by user input. The following steps demonstrate how to achieve this:

1. **Take User Input:** We first ask the user to input the number of rows and columns for the matrix. This allows flexibility in creating matrices of different sizes.
2. **Create and Initialize the Matrix:** Using the provided dimensions, we create a matrix with all elements initialized to zero.
3. **Print the Matrix:** Finally, we print the initialized matrix to verify the result.

Example Code:

```
# Take user input for number of rows and columns
num_rows = int(input("Enter the number of rows: "))
num_columns = int(input("Enter the number of columns: "))

# Initialize the result matrix with zeros
result = []

# Create each row of the result matrix
for i in range(num_rows):
    # Create a row with zeros
    row = []
    for j in range(num_columns):
        row.append(0)
    # Add the row to the result matrix
    result.append(row)

# Now result is a matrix with the same dimensions as specified, filled with zeros
print("Initialized matrix with zeros:")
for row in result:
    print(row)
```

Listing 3: Matrix Initialization with User Input

Explanation:

- **User Input:** The number of rows and columns are taken from user input using the `input()` function. These inputs are converted to integers using `int()`.
- **Matrix Initialization:**
  - An empty list `result` is created to hold the rows of the matrix.
  - A loop iterates over the number of rows. For each row, another empty list `row` is created.
  - Inside the inner loop, zeros are appended to `row` for each column.
  - After creating a row, it is added to the `result` matrix.
- **Print the Matrix:** - The initialized matrix is printed row by row to verify that it has been created correctly.

Dry Run:

- **Assumptions:** Assume the user inputs `3` for rows and `4` for columns.
- **Initialization:**
  - `num_rows = 3`

  - `num_columns = 4`
  - `result = []`

- **Iteration:**
  - For `i = 0`:
    - `row = []`
    - Inner loop iterates `j = 0` to `3`:
      - `row = [0]` (for `j = 0`)
      - `row = [0, 0]` (for `j = 1`)
      - `row = [0, 0, 0]` (for `j = 2`)
      - `row = [0, 0, 0, 0]` (for `j = 3`)
    - `result = [[0, 0, 0, 0]]`
  - For `i = 1`:
    - `row = []`
    - Inner loop iterates `j = 0` to `3`:
      - `row = [0]` (for `j = 0`)
      - `row = [0, 0]` (for `j = 1`)
      - `row = [0, 0, 0]` (for `j = 2`)
      - `row = [0, 0, 0, 0]` (for `j = 3`)
    - `result = [[0, 0, 0, 0], [0, 0, 0, 0]]`
  - For `i = 2`:
    - `row = []`
    - Inner loop iterates `j = 0` to `3`:
      - `row = [0]` (for `j = 0`)
      - `row = [0, 0]` (for `j = 1`)
      - `row = [0, 0, 0]` (for `j = 2`)
      - `row = [0, 0, 0, 0]` (for `j = 3`)
    - `result = [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]`

- **Result:** The final `result` matrix is:

```
    [[0, 0, 0, 0],
     [0, 0, 0, 0],
     [0, 0, 0, 0]]

```

2.4 Matrix Addition Example

```
# Example matrices
A = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

B = [
    [9, 8, 7],
    [6, 5, 4],
    [3, 2, 1]
```

```
]

# Initialize result matrix with zeros
result = [[0 for _ in range(len(A[0]))] for _ in range(len(A))]

# Perform addition
for i in range(len(A)):
    for j in range(len(A[0])):
        result[i][j] = A[i][j] + B[i][j]

# Print the result
print("Matrix A:")
for row in A:
    print(row)

print("\nMatrix B:")
for row in B:
    print(row)

print("\nMatrix Addition Result:")
for row in result:
    print(row)
```

Listing 4: Matrix Addition Example Without Function

Dry Run:

- **Matrices:**
  - `A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`
  - `B = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]`
- **Initialization:**
  - `result` is initialized as a 3x3 matrix with all elements set to 0.
- **Iteration:**
  - For `i = 0`:
    - For `j = 0`: `result[0][0] = A[0][0] + B[0][0]` → `result[0][0] = 1 + 9 = 10`
    - For `j = 1`: `result[0][1] = A[0][1] + B[0][1]` → `result[0][1] = 2 + 8 = 10`
    - For `j = 2`: `result[0][2] = A[0][2] + B[0][2]` → `result[0][2] = 3 + 7 = 10`
  - For `i = 1`:
    - For `j = 0`: `result[1][0] = A[1][0] + B[1][0]` → `result[1][0] = 4 + 6 = 10`
    - For `j = 1`: `result[1][1] = A[1][1] + B[1][1]` → `result[1][1] = 5 + 5 = 10`
    - For `j = 2`: `result[1][2] = A[1][2] + B[1][2]` → `result[1][2] = 6 + 4 = 10`
  - For `i = 2`:
    - For `j = 0`: `result[2][0] = A[2][0] + B[2][0]` → `result[2][0] = 7 + 3 = 10`
    - For `j = 1`: `result[2][1] = A[2][1] + B[2][1]` → `result[2][1] = 8 + 2 = 10`
    - For `j = 2`: `result[2][2] = A[2][2] + B[2][2]` → `result[2][2] = 9 + 1 = 10`
- **Result:** `result = [[10, 10, 10], [10, 10, 10], [10, 10, 10]]`

2.5 Matrix Multiplication

Matrix multiplication involves multiplying the rows of the first matrix with the columns of the second matrix.

```python
# Matrix Multiplication
rows_A = int(input("Enter the number of rows for Matrix A: "))
cols_A = int(input("Enter the number of columns for Matrix A: "))
rows_B = int(input("Enter the number of rows for Matrix B: "))
cols_B = int(input("Enter the number of columns for Matrix B: "))

# Check if the matrices can be multiplied
if cols_A != rows_B:
    print("Error: Number of columns in Matrix A must be equal to the number of rows in Matrix B.")
else:
    # Initialize matrices
    A = []
    B = []

    # Input Matrix A
    print("Enter the elements of Matrix A row by row:")
    for i in range(rows_A):
        row = []
        for j in range(cols_A):
            element = int(input(f"Enter element A[{i}][{j}]: "))
            row.append(element)
        A.append(row)

    # Input Matrix B
    print("Enter the elements of Matrix B row by row:")
    for i in range(rows_B):
        row = []
        for j in range(cols_B):
            element = int(input(f"Enter element B[{i}][{j}]: "))
            row.append(element)
        B.append(row)

    # Initialize result matrix with zeros
    result = []
    for i in range(rows_A):
        result.append([0] * cols_B)

    # Perform multiplication
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    # Printing the result
    print("Resultant Matrix:")
    for row in result:
        print(row)
```

Listing 5: Matrix Multiplication Example

Dry Run: Let's walk through a dry run of the code with an example:
User Input

- Number of rows for Matrix A: 2
- Number of columns for Matrix A: 2
- Number of rows for Matrix B: 2

- Number of columns for Matrix B: 2
- Elements of Matrix A:
  - First row: 1 2
  - Second row: 3 4
- Elements of Matrix B:
  - First row: 5 6
  - Second row: 7 8

Dry Run

- **Initialization:**
  - `rows_A = 2, cols_A = 2`
  - `rows_B = 2, cols_B = 2`
  - `A = []`
  - `B = []`
- **Input Matrix A:**
  - For `i = 0`:
    - For `j = 0: element = 1, row = [1]`
    - For `j = 1: element = 2, row = [1, 2]`
    - `A = [[1, 2]]`
  - For `i = 1`:
    - For `j = 0: element = 3, row = [3]`
    - For `j = 1: element = 4, row = [3, 4]`
    - `A = [[1, 2], [3, 4]]`
- **Input Matrix B:**
  - For `i = 0`:
    - For `j = 0: element = 5, row = [5]`
    - For `j = 1: element = 6, row = [5, 6]`
    - `B = [[5, 6]]`
  - For `i = 1`:
    - For `j = 0: element = 7, row = [7]`
    - For `j = 1: element = 8, row = [7, 8]`
    - `B = [[5, 6], [7, 8]]`
- **Initialize Result Matrix:**
  - `result = [[0, 0], [0, 0]]`
- **Perform Multiplication:**
  - For `i = 0`:
    - For `j = 0`:

        - For `k = 0`: `result[0][0] += A[0][0] * B[0][0]` → `result[0][0] = 0 + (1 * 5) = 5`
        - For `k = 1`: `result[0][0] += A[0][1] * B[1][0]` → `result[0][0] = 5 + (2 * 7) = 19`
    - For `j = 1`:
        - For `k = 0`: `result[0][1] += A[0][0] * B[0][1]` → `result[0][1] = 0 + (1 * 6) = 6`
        - For `k = 1`: `result[0][1] += A[0][1] * B[1][1]` → `result[0][1] = 6 + (2 * 8) = 22`
- For `i = 1`:
    - For `j = 0`:
        - For `k = 0`: `result[1][0] += A[1][0] * B[0][0]` → `result[1][0] = 0 + (3 * 5) = 15`
        - For `k = 1`: `result[1][0] += A[1][1] * B[1][0]` → `result[1][0] = 15 + (4 * 7) = 43`
    - For `j = 1`:
        - For `k = 0`: `result[1][1] += A[1][0] * B[0][1]` → `result[1][1] = 0 + (3 * 6) = 18`
        - For `k = 1`: `result[1][1] += A[1][1] * B[1][1]` → `result[1][1] = 18 + (4 * 8) = 50`
- **Final Result Matrix:**
    - `result = [[19, 22], [43, 50]]`
- **Output:** The final result matrix is printed:

```
    [19, 22]
    [43, 50]

```

- **Summary:**
    - **Initialization:** Start with empty matrices A and B and an empty result matrix.
    - **Input Matrices:** Read each element for both matrices from the user and populate the matrices.
    - **Multiplication:** Perform matrix multiplication by iterating through the rows of A and columns of B, and updating the result matrix accordingly.
    - **Output:** Print the final result matrix.

3 Diagonal Sum

The diagonal sum of a matrix involves summing the elements on the primary and secondary diagonals.

3.1 Primary Diagonal Sum

The primary diagonal runs from the top-left to the bottom-right of the matrix.

```
def primary_diagonal_sum(matrix):
    sum_diag = 0
    for i in range(len(matrix)):
        sum_diag += matrix[i][i]
```

```
    return sum_diag

# Example matrix
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print("Primary Diagonal Sum:", primary_diagonal_sum(matrix))
```

Listing 6: Primary Diagonal Sum Example

Dry Run:

- **Matrix:**
  - `matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`
- **Initialization:**
  - `sum_diag = 0`
- **Iteration:**
  - For `i = 0`: `sum_diag += matrix[0][0]` → `sum_diag = 0 + 1 = 1`
  - For `i = 1`: `sum_diag += matrix[1][1]` → `sum_diag = 1 + 5 = 6`
  - For `i = 2`: `sum_diag += matrix[2][2]` → `sum_diag = 6 + 9 = 15`
- **Result:** `Primary Diagonal Sum = 15`

3.2 Secondary Diagonal Sum

The secondary diagonal runs from the top-right to the bottom-left of the matrix.

```
def secondary_diagonal_sum(matrix):
    sum_diag = 0
    n = len(matrix)
    for i in range(n):
        sum_diag += matrix[i][n - 1 - i]
    return sum_diag

# Example matrix
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print("Secondary Diagonal Sum:", secondary_diagonal_sum(matrix))
```

Listing 7: Secondary Diagonal Sum Example

Dry Run:

- **Matrix:**
  - `matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`
- **Initialization:**
  - `sum_diag = 0`
  - `n = 3`
- **Iteration:**
  - For `i = 0`: `sum_diag += matrix[0][2]` → `sum_diag = 0 + 3 = 3`

- For `i = 1: sum_diag += matrix[1][1]` → `sum_diag = 3 + 5 = 8`
- For `i = 2: sum_diag += matrix[2][0]` → `sum_diag = 8 + 7 = 15`

* **Result:** `Secondary Diagonal Sum = 15`

4 Dictionaries

Dictionaries in Python are collections of key-value pairs. Each key is unique and is used to access its associated value.

4.1 Example 1: Basic Operations

```python
# Create a dictionary
student_scores = {
    "Alice": 85,
    "Bob": 92,
    "Charlie": 78
}

# Add a new key-value pair
student_scores["David"] = 90

# Update an existing key
student_scores["Alice"] = 88

# Access a value by key
alice_score = student_scores["Alice"]

# Remove a key-value pair
del student_scores["Charlie"]

# Print the dictionary
print(student_scores)
```

Listing 8: Basic Dictionary Operations

Dry Run:

* **Initial Dictionary:**
  - `student_scores = {"Alice": 85, "Bob": 92, "Charlie": 78}`
* **Add New Key-Value Pair:**
  - `student_scores["David"] = 90`
  - New dictionary: `{"Alice": 85, "Bob": 92, "Charlie": 78, "David": 90}`
* **Update Existing Key:**
  - `student_scores["Alice"] = 88`
  - Updated dictionary: `{"Alice": 88, "Bob": 92, "Charlie": 78, "David": 90}`
* **Access Value by Key:**
  - `alice_score = student_scores["Alice"]` → `alice_score = 88`
* **Remove Key-Value Pair:**
  - `del student_scores["Charlie"]`
  - Final dictionary: `{"Alice": 88, "Bob": 92, "David": 90}`

4.2 Example 2: Nested Dictionaries

```python
# Create a nested dictionary
class_info = {
    "ClassA": {
        "Teacher": "Mr. Smith",
        "Students": ["Alice", "Bob", "Charlie"]
    },
    "ClassB": {
        "Teacher": "Ms. Johnson",
        "Students": ["David", "Eva", "Frank"]
    }
}

# Access nested values
class_a_teacher = class_info["ClassA"]["Teacher"]
class_b_students = class_info["ClassB"]["Students"]

# Print values
print("Class A Teacher:", class_a_teacher)
print("Class B Students:", class_b_students)
```

Listing 9: Nested Dictionary Example

Dry Run:

* **Initial Nested Dictionary:**
  - `class_info = {"ClassA": {"Teacher": "Mr. Smith", "Students": ["Alice", "Bob", "Charlie"]}, "ClassB": {"Teacher": "Ms. Johnson", "Students": ["David", "Eva", "Frank"]}}`
* **Access Nested Values:**
  - `class_a_teacher = class_info["ClassA"]["Teacher"]` → `class_a_teacher = "Mr. Smith"`
  - `class_b_students = class_info["ClassB"]["Students"]` → `class_b_students = ["David", "Eva", "Frank"]`
* **Output:**
  - `"Class A Teacher: Mr. Smith"`
  - `"Class B Students: ['David', 'Eva', 'Frank']"`

4.3 Example 3: Dictionary Comprehensions

```python
# Create a dictionary using comprehension
squares = {x: x**2 for x in range(1, 6)}

# Print the dictionary
print(squares)
```

Listing 10: Dictionary Comprehension Example

Dry Run:

* **Dictionary Comprehension:**
  - For `x = 1`: `1: 1**2 = 1`
  - For `x = 2`: `2: 2**2 = 4`
  - For `x = 3`: `3: 3**2 = 9`
  - For `x = 4`: `4: 4**2 = 16`
  - For `x = 5`: `5: 5**2 = 25`

PREPARED BY: Sahil Wagh(M.Tech),Nitish Dumoliya (Phd)

- **Resulting Dictionary:**
  - `squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}`

5 Functions in Python

Functions are a way to encapsulate code into reusable blocks. In Python, functions are defined using the `def` keyword.

5.1 Example 1: Function to Find the Maximum of Two Numbers

```
def find_max(a, b):
    if a > b:
        return a
    else:
        return b

# Calling the function and printing the result
result = find_max(10, 20)
print("Maximum:", result)
```

Listing 11: Find the Maximum of Two Numbers

Dry Run Explanation:

- **Function Definition:**
  - `def find_max(a, b):` defines a function named `find_max` that takes two parameters `a` and `b`.
- **Function Call:**
  - `result = find_max(10, 20)` calls the function with `a = 10` and `b = 20`.
- **Execution:**
  - The function checks if `a > b`.
  - Since `10 > 20` is `False`, it returns `b`, which is `20`.
- **Output:**
  - The returned value `20` is stored in `result`.
  - `print("Maximum:", result)` prints `Maximum: 20`.

5.2 Example 2: Function to Calculate the Sum of a List

```
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total

# Calling the function and printing the result
result = sum_list([1, 2, 3, 4, 5])
print("Sum:", result)
```

Listing 12: Calculate the Sum of a List

Dry Run Explanation:

- **Function Definition:**

- `def sum_list(numbers):` defines a function named `sum_list` that takes a list `numbers`.

* **Function Call:**
  - `result = sum_list([1, 2, 3, 4, 5])` calls the function with `numbers = [1, 2, 3, 4, 5]`.
* **Execution:**
  - `total` is initialized to `0`.
  - The loop iterates over each element in the list:
  - `total += 1` → `total = 1`
  - `total += 2` → `total = 3`
  - `total += 3` → `total = 6`
  - `total += 4` → `total = 10`
  - `total += 5` → `total = 15`
  - The function returns `total`, which is `15`.
* **Output:**
  - The returned value `15` is stored in `result`.
  - `print("Sum:", result)` prints `Sum:  15`.

5.3 Example 3: Function to Check if a Number is Even or Odd

```python
def is_even(number):
    if number % 2 == 0:
        return True
    else:
        return False

# Calling the function and printing the result
result = is_even(7)
print("Is 7 even?:", result)
```

Listing 13: Check if a Number is Even or Odd

Dry Run Explanation:

* **Function Definition:**
  - `def is_even(number):` defines a function named `is_even` that takes a parameter `number`.
* **Function Call:**
  - `result = is_even(7)` calls the function with `number = 7`.
* **Execution:**
  - The function checks if `number % 2 == 0`.
  - Since `7 % 2 == 0` is `False`, it returns `False`.
* **Output:**
  - The returned value `False` is stored in `result`.
  - `print("Is 7 even?:", result)` prints `Is 7 even?:  False`.

End of Lab

Thank you for completing the lab session. Please ensure you have understood all the concepts and completed the exercises. If you have any questions, feel free to ask your instructor or teaching assistant.

PREPARED BY: Sahil Wagh(M.Tech),Nitish Dumoliya (Phd)